Examen Taller de base de datos **de** Sesion 01 **Recuperación**

El docente me entregó las preguntas de estudio desde el 24 de agosto. Adicionalmente, con el fin de evitar que desapruebe la asignatura debido a mis bajas calificaciones y a la falta de entrega oportuna de mis tareas, se me otorgó un plazo extraordinario de 4 meses para resolver y entregar estas preguntas. Reconozco que el docente me ha brindado todas las facilidades académicas necesarias, incluyendo el código base para el desarrollo de cada trabajo. Por ello, la nota que obtenga en esta evaluación será de mi entera y única responsabilidad, reflejo directo de mi falta de interés y compromiso con el curso, dado que hasta la fecha esto es todo lo que he logrado avanzar

Nombres y Apellidos: Fernando Huel Condori Huamani **DNI:** 60427249

• Presentación y legibilidad: Buena caligrafía y formato claro. • Estructura del código: Todas las líneas se encuentran debidamente numeradas. • Implementación: Incluye el código fuente solicitado. • Avance y entregas: Registra la totalidad de las sesiones completadas. • Control de versiones: Cuenta con todo el repositorio actualizado y alojado en GitHub. • Agregó 10 preguntas al final

1 Sesión 4: Crud con pHP y Mysql (super tienda). crud significa Crear,
2 leer, actualizar y eliminar registros actualizar y eliminar datos. lo hicimos
3 con php y Mysql para la super tienda. primero, index.php se conecta a la base
4 de datos con conexion.php. cuando el usuario llena el formulario y
5 presiona registrar, PHP revisa que los datos esten bien (que no
6 esten vacios y que precio y stock sean numeros) y recien ahi los
7 guarda en la tabla productos. para guardar usamos consultas preparadas
8 por que asi nadie puede meter codigo malicioso (inyeccion sql). la
9 despues la pagina muestra los productos en una tabla y uso
10 html specialchars para que el texto no se ejecute como codigo.
11 Ademas hicimos tres retos: un buscador por nombre, una alerta si el
12 stock es menor a 5 y el valor total del inventario

```
13   require_once 'conexion.php';
14   $r = $conexion->query(
15   "select sum(precio*stock) as total from productos");
16   $valor_inventario = $r->fetch_assoc()['total'] ?? 0;
17   if ($_server["REQUEST_METHOD"] === "POST"){
18   $nombre = trim($_post["nombre"] ?? "");
19   $categoria = trim($_post["categoria"] ?? "");
20   $precio = $_post["precio"] ?? ""; $stock = $_post["stock"] ?? "";
21   if($nombre !== "" && $categoria !== "" && $categoria !== "" && is_numeric($precio)
22   && $precio >= 0
23   && filter_var($stock, filter_validate_int) !== false){
24   $sql = "insert into productos (nombre, categoria, precio, stock)
25       values (?, ?, ?, ?)";
26   $stmt = $conexion->prepare($sql);
27   $stmt->bind_param("ssdi", $nombre, $categoria, $precio, $stock);
28   $stmt->execute();
29   header("location: index.php"); exit; }}
30   $buscar = trim($_GET['buscar'] ?? '');
31   Sql = "select * from productos where nombre like ? ORDER BY id DESC";
32   $parametro = "%" . $buscar . "%";
33   <tr class="<?= ($fila['stock'] < 5) ? 'fila-alerta' : '' ?>">
34   <a href="editar.php?id=<?= $fila["id"] ?>">editar</a>
35
```

36 ¿Que significa crud y para que sirve?
37 ¿Que hace require_once "conexion.php" al inicio del archivo?
38 ¿Por que usamos prepare() y no ponemos los datos directos en el SQL?
39 En bind_param("ssdi") ¿que tipo de dato es cada letra?
40 ¿Para que sirve header("location: index.php") y exit?
41 ¿Que problema evita htmlspecialchars() al mostrar los datos?
42 ¿Como se calcula el valor total del inventario con SQL
43 ¿Para que se usan los simbolos % en el buscador con like?
44 ¿Cuando sale la alerta de stock bajo en una fila de la tabla
45 ¿Por que el boton eliminar pide confirmacion antes de borrar?

Elabore un resumen de la Sesión 4 destacando los aspectos más relevantes y los temas principales abordados

Seleccione los trabajos terminados que ya se encuentran subidos a su repositorio:

- ☐ **Sección 1**..... echo
- ☐ **Sección 2**..... echo
- ☐ **Sección 3**..... echo
- ☐ **Sección 4**..... echo
- ☐ **Sección 5**..... echo
- ☐ **Sección 6**..... echo
- ☐ **Sección 7**.....
- ☐ **Sección 8**.....

Examen Taller de base **de** datos Sesion-02 **Recuperació**

El docente me entregó las preguntas de estudio desde el 24 de agosto. Adicionalmente, con el fin de evitar que desapruebe la asignatura debido a mis bajas calificaciones y a la falta de entrega oportuna de mis tareas, se me otorgó un plazo extraordinario de 4 meses para resolver y entregar estas preguntas. Reconozco que el docente me ha brindado todas las facilidades académicas necesarias, incluyendo el código base para el desarrollo de cada trabajo. Por ello, la nota que obtenga en esta evaluación será de mi entera y única responsabilidad, reflejo directo de mi falta de interés y compromiso con el curso, dado que hasta la fecha esto es todo lo que he logrado avanzar

Nombres y Apellidos: Fernando Noel Condor Huaman **DNI:** 60427799 -4-

• Presentación y legibilidad: Buena caligrafía y formato claro. • Estructura del código: Todas las líneas se encuentran debidamente numeradas. • Implementación: Incluye el código fuente solicitado. • Avance y entregas: Registra la totalidad de las sesiones completadas. • Control de versiones: Cuenta con todo el repositorio actualizado y alojado en GitHub. • Agregó 10 preguntas al final

1 En la sesión 2 seguimos trabajando con la base juego_online que creamos
2 en la sesion 1. Esta vez no se trata de crear tablas, sino entender
3 de como se organiza un servidor de datos y quien puede
4 hacer dentro de el. Todo se ejecuto en phpMyAdmin en la pestaña
5 SQL, con MySQL 8.0 o MariaDB 10.5 o superior
6 El primer script es mas una nota que un programa: casi todos son
7 comentarios, por eso al ejecutarlo no se ningun cambio. Sirve para
8 dejar documentado que se eligio MySQL o MariaDB, que la arquitectura
9 tiene un servidor primario con replicas de lectura por region y algunos
10 parametros recomendados, como usar el 75% de la ram en innodb buffer
11 pool_size, poner max_connections de 100 y activar los_bin para los
12 respaldos.
13 El segundo script es el mas importante: crea tres roles con distintos
14 niveles de poder. El DBA es el administrador y tiene control total,
15 es como el dueño del servidor. El SA es soporte de infraestructura:
16 puede ver los procesos y el estado del servidor pero no puede leer
17 los datos de jugadores ni de partidas. El DA es el analista y solo puede
18 hacer select en las tablas jugadores, partidas y logros, asi que ni
19 por error puede borrar o modificar algo.
20
21 Todo esto se basa en principio de minimo privilegio: cada cuenta
22 recibe solo el acceso que necesita para su trabajo, ni mas ni menos.
23 En la vida real es muy importante, por que normas como PCI-DSS y SOX
24 piden separar funciones y casos como la filtracion de Capital One en 2019
25 se dieron por permisos demasiados amplios.
26
27 Las contraseñas de los scripts dicen cambiar en produccion 123 a proposito;
28 es un recordatorio de que se deben cambiar antes de usar el sistema
29 fuera del aula. Al final se verifico con show grants que analista_da
30 solo tenga select

```
31 CREATE USER 'dba_admin'@'%' IDENTIFIED BY 'cambiar en produccion123';
32 GRANT ALL privileges ON juego_online.* TO 'dba_admin'@'%'
33    WITH GRANT OPTION.
34 GRANT PROCES, RELOAD, REPLICATION CLIENT ON *.* TO 'soporte_sa'@'%';
```

35 ¿Por que base de datos se trabajo en la sesion 2?
36 ¿Por que el primer script no muestra cambios al ejecutarse?
37 ¿Que arquitectura se documenta y para que sirven las replicas?
38 ¿Que significan las siglas DBA, SA y DA?
39 ¿Que puede hacer el rol dba que los otros no pueden?
40 ¿Que puede hacer el rol SA y que datos no puede leer?
41 ¿Por que el rol DA solo tiene permiso select?
42 ¿Que es el principio de minimo privilegio? De un ejemplo
43 ¿Para que sirve el comando SHOW GRANTS?
44 ¿Por que todas las contraseñas dicen cambiar en produccion 123?

Elabore un resumen de la Sesión 5 destacando los aspectos más relevantes y los temas principales abordados

1.

Seleccione los trabajos terminados que ya se encuentran subidos a su repositorio:

- [ ] Sección 1
- [ ] Sección 2
- [ ] Sección 3
- [ ] Sección 4
- [ ] Sección 5
- [ ] Sección 6 echo
- [ ] Sección 7
- [ ] Sección 8

Taller de base de datos sesion_03 **Recuperación**

Examen

El docente me entregó las preguntas de estudio desde el 24 de agosto. Adicionalmente, con el fin de evitar que desapruebe la asignatura debido a mis bajas calificaciones y a la falta de entrega oportuna de mis tareas, se me otorgó un plazo extraordinario de 4 meses para resolver y entregar estas preguntas. Reconozco que el docente me ha brindado todas las facilidades académicas necesarias, incluyendo el código base para el desarrollo de cada trabajo. Por ello, la nota que obtenga en esta evaluación será de mi entera y única responsabilidad, reflejo directo de mi falta de interés y compromiso con el curso, dado que hasta la fecha esto es todo lo que he logrado avanzar

Nombres y Apellidos: Condori Huaman Fernanda Ruth ....**DNI:** 60437391 ......-3-

• Presentación y legibilidad: Buena caligrafía y formato claro. • Estructura del código: Todas las líneas se encuentran debidamente numeradas. • Implementación: Incluye el código fuente solicitado. • Avance y entregas: Registra la totalidad de las sesiones completadas. • Control de versiones: Cuenta con todo el repositorio actualizado y alojado en GitHub. • Agregó 10 preguntas al final

1. En la sesion 3, siguimos con la base de datos juegos - online, pero ahora
2. el tema fue el trafico masivo y la concurrencia, es decir, que pasa cuando muchos jugadores usan el servidor al mismo tiempo. Se trabajo en php y admin con tablas innodb y, para la simulacion, con dos pestañas del navegador abiertas a la vez.

En el primer paso agregamos dos columnas nuevas a la tabla items: es_unico y vendido para marcar un item especial de lanzamiento que solo se puede vender una vez en todo el juego: la skin dragon dorado. Lo importante es que, en innodb, agregar una columna con default es una operación que no bloquea la tabla, o sea que el juego sigue funcionando mientras se hace el cambio.

El segundo paso fue crear el procedimiento comprar_item_unico y resuelve la condición de carrera: si dos jugadores compran el mismo item unico al mismo tiempo, los dos pueden leer que vendido es false y se venderia dos veces, como el ultimo plato del dia que dos clientes piden a la vez. Para evitarlo se uso START TRANSACTION y SELECT ... FOR UPDATE, que pone un candado en esa fila hasta el commit o rollback. Asi el segundo jugador espera su turno y, cuando le toca, ya ve vendido en true y lo rechazan.

Las transacciones tambien sirven para que todo se haga junto o no se haga nada (atomicidad): no se cobra sin entregar el item. En el paso 3 probamos con dos pestañas, una para el jugador 2 y otro para el 5, y solo una mostro exito y el otro rechazado. Tambien vimos el deadlock cuando dos transacciones se esperan entre si; MySQL lo detecta y cancela una, por eso la aplicacion debe reintentar.

Esto no solo es teoria: empresas como GitHub cambian tablas enormes sin parar el sistema y en lanzamientos como el de Ticketmaster en 2022 el mal control de la concurrencia causo ventas dobles.

```
Alter tabla items
ADD column if NOT exists es_unico Booleam NOT null default false;
update items set es_unico = true where nombre = 'skin dragon dorado';
start transaction
select vendido into v_vendido from items where item_id = p_item_id
for update;
```

- ¿Que dos columnas se agregaron a la tabla items en el paso 1?
- ¿Por que agregar una columna con default no bloqueo la tabla?
- ¿Que es una condicion de carrera? Explicalo con un ejemplo
- ¿Para que sirve select for update?
- ¿Que hacen start transaction, commit y rollback?
- ¿Que significa atomicidad en una compra?
- ¿Por que la segunda pestaña espera y no da error?
- ¿Que resultado se espera al ejecutar las dos pestañas a la vez?
- ¿Que es un deadlock y que hace MySQL cuando ocurre?
- ¿Por que necesitan dos pestañas del navegador en el paso 3?

Elabore un resumen de la Sesión 4 destacando los aspectos más relevantes y los temas principales abordados

Seleccione los trabajos terminados que ya se encuentran subidos a su repositorio:

- **[ ] Sección 1**
- **[ ] Sección 2**
- **[ ] Sección 3**
- **[ ] Sección 4**
- **[ ] Sección 5**
- **[ ] Sección 6** echo
- **[ ] Sección 7**
- **[ ] Sección 8**

Examen taller de base **de** datos sesion_04 **Recupera**

El docente me entregó las preguntas de estudio desde el 24 de agosto. Adicionalmente, con el fin de evitar que desapruebe la asignatura debido a mis bajas calificacio y a la falta de entrega oportuna de mis tareas, se me otorgó un plazo extraordinario de 4 meses para resolver y entregar estas preguntas. Reconozco que el docente me brindado todas las facilidades académicas necesarias, incluyendo el código base para el desarrollo de cada trabajo. Por ello, la nota que obtenga en esta evaluación sera de mi entera y única responsabilidad, reflejo directo de mi falta de interés y compromiso con el curso, dado que hasta la fecha esto es todo lo que he logrado avanzar

Nombres y Apellidos: Fernando Noel Codor Huaman **DNI:** 60923329 -4-

• Presentación y legibilidad: Buena caligrafía y formato claro. • Estructura del código: Todas las líneas se encuentran debidamente numeradas. • Implementación: Incluye el código fuente solicitado. • Avance y entregas: Registra la totalidad de las sesiones completadas. • Control de versiones: Cuenta con todo el repositorio actualizado y alojado en GitHub. • Agregó 10 preguntas al final

1. En esta sesión aprendimos la base de datos del juego de tres formas
2. distintas.

Primero vimos las restricciones check. Se usa para que la base de datos mismo rechaze valores que no tienen sentido como monedas negativas, experiencia negativa o un nivel menor a 1. También en la tabla de partidas el puntaje no pueda ser negativo y el resultado solo puede ser victoria, derrota o jugado. La idea es que si alguien intente hacer trampa desde el cliente, el motor no deja guardar esos datos

Después vimos los Roles de aplicación

El rol_jugador: Puede ver información, registrar sus partidas, usar los procedimientos de subir de nivel y comprar items, pero no puede modificar monedas ni niveles a mano.

El rol_moderador hereda todo lo del jugador además puede banear

El rol_administrador hereda todo y es el único que puede cambiar monedas

Esto se llama privilegio mínimo: cada uno tiene lo que necesita.

Por último vimos la inyección SQL si el backend junta el texto que escribe el usuario directamente en la consulta un atacante puede meter código y sacar todos los datos o hacer cambios la solución es usar un procedimiento con parametro (buscar_jugador_seguro) o consultas preparadas, para que el texto siempre se trate como dato y nunca como código.

En resumen
la sesión trata de poner reglas dentro de la base de datos, controlar bien los permisos de cada tipo de usuario y no dejar que se inyecte SQL

Para que sirve las restricciones check?
Que valores no se permiten en monedas y nivel?
Que puede hacer el rol_jugador?
Que permisos extras tiene el moderador?
Quien puede modificar monedas directamente?
Que significa privilegio minimo?
Que es inyección SQL?
Por que es peligroso concatenar el texto del usuario?
Que hace el procedimiento buscar_jugador_seguro?
Que tecnica uso para evitar inyecciones SQL?

re un resumen de la Sesión 5 destacando los aspectos más relevantes y los temas principales abordados

cione los trabajos terminados que ya se encuentran subidos a su repositorio:

- [ ] **Sección 1**
- [ ] **Sección 2**
- [ ] **Sección 3**
- [ ] **Sección 4**
- [ ] **Sección 5**
- [ ] **Sección 6** echo
- [ ] **Sección 7**

Examen Taller de base **de** datos sesion-05 **Recuperación**

El docente me entregó las preguntas de estudio desde el 24 de agosto. Adicionalmente, con el fin de evitar que desapruebe la asignatura debido a mis bajas calificaciones y a la falta de entrega oportuna de mis tareas, se me otorgó un plazo extraordinario de 4 meses para resolver y entregar estas preguntas. Reconozco que el docente me ha rindado todas las facilidades académicas necesarias, incluyendo el código base para el desarrollo de cada trabajo. Por ello, la nota que obtenga en esta evaluación será e mi entera y única responsabilidad, reflejo directo de mi falta de interés y compromiso con el curso, dado que hasta la fecha esto es todo lo que he logrado avanzar

Nombres y Apellidos: Condori Huamani Fernando Noel **DNI:** 60427299 **-3-**

Presentación y legibilidad: Buena caligrafía y formato claro. • Estructura del código: Todas las líneas se encuentran debidamente numeradas. • mplementación: Incluye el código fuente solicitado. • Avance y entregas: Registra la totalidad de las sesiones completadas. • Control de versiones: uenta con todo el repositorio actualizado y alojado en GitHub. • Agregó 10 preguntas al final

En esta sesion aprendimos a manejar grandes volumenes de datos separandolos por region y cargando miles de registros de una sola vez. Primero vimos la fragmentación. Se creo la tablas partidos se guarda en el fragmento de su region como si fueron dos servidores distintos. cuando se busco solo por una region, ya no revisa el otro fragmento (eso se llamo partition pruning) Tambien se aprendio que las tablas particionadas no permiten claves foraneas, por eso la integridad se controla desde la aplicación

Despues se crearon indices para que el matchmaking sea rapido. sin indice el motor tien revisar todas las filas y si se vuelve lento. Con indice busca solo lo necesario y responde en milesegundos. se hizo un indice por region y fecha, otro por jugador y otro por region y nivel de los jugadores.

Por ultimo se hizo una carga masiva Tipo ETL. se ~~hizo~~ uso un CTE recursivo para insertar 5000 partidas de golpe simulando la importacion de un archivo grande. Tambien se explico como se haria lo mismo con load data infile o subiendo y bajando archivos desde la nube.

Tambien en la sesion se hablo sobre escalar la base de datos separando por region o acelerar las busquedas con indices y cargar muchos datos de forma reciente

Preguntas

Que es la fragmentacion por region?

Para que sirve la partition pruning

Por que no se puede usar claves foraneas en las tablas particionadas?

Que problemas soluciona los indices?

Que columnas se usaron para el indice del matchmaking?

Que significa las siglas ETL?

como se generaron las 5000 partidas en el ejercicio?

Que es LOAD DATA INFILE?

Por que se separaron los datos por region en juegos reales?

Que ventaja tiene cargar datos por lotes en vez de uno por uno?

labore un resumen de la Sesión 4 destacando los aspectos más relevantes y los temas principales abordados

leccione los trabajos terminados que ya se encuentran subidos a su repositorio:

- ☐ **Sección 1**
- ☐ **Sección 2**
- ☐ **Sección 3**
- ☐ **Sección 4**
- ☐ **Sección 5** hecho
- ☐ **Sección 6**
- ☐ **Sección 7**

Examen Taller de base **de** datos sesión - 06 **Recuperación**

El docente me entregó las preguntas de estudio desde el 24 de agosto. Adicionalmente, con el fin de evitar que desapruebe la asignatura debido a mis bajas calificacion y a la falta de entrega oportuna de mis tareas, se me otorgó un plazo extraordinario de 4 meses para resolver y entregar estas preguntas. Reconozco que el docente me ha brindado todas las facilidades académicas necesarias, incluyendo el código base para el desarrollo de cada trabajo. Por ello, la nota que obtenga en esta evaluación será de mi entera y única responsabilidad, reflejo directo de mi falta de interés y compromiso con el curso, dado que hasta la fecha esto es todo lo que he logrado avanzar

Nombres y Apellidos: Condori Huaman Facundo Eliel **DNI:** 60427349 -4-

• Presentación y legibilidad: Buena caligrafía y formato claro. • Estructura del código: Todas las líneas se encuentran debidamente numeradas. • Implementación: Incluye el código fuente solicitado. • Avance y entregas: Registra la totalidad de las sesiones completadas. • Control de versiones: Cuenta con todo el repositorio actualizado y alojado en GitHub. • Agregó 10 preguntas al final

1. Respaldos y recuperación anti ataques
2. En esta sesión aprendimos a proteger la base de datos ante fallos o ataques y a recuperarla si se pierde

Primero se creó trigger de auditoría. Cada vez que alguien cambió las monedas de un jugador, el sistema guardó automáticamente el valor anterior, el nuevo, quién lo hizo y cuándo. Así quedó un historial de cambios importantes.

Después se hizo un respaldo total con un script. El comando mysqldump generó un archivo .sql con toda la base de datos, incluyendo tablas, procedimiento y triggers. El archivo se guardó con fecha y hora para saber cuál es el más reciente

También se explicó el respaldo incremental con binary logs (binlog). Este registro todos los cambios que se hacen después del último respaldo completo. Si hay un problema, se puede restaurar el respaldo total y luego solo aplicar los cambios hasta momento del incidente (eso se llama PITR)

Por último se el simulacro de desastre
se borró la base de datos a propósito y se recuperó desde
el último respaldo. Si el respaldo estaba actualizado completos.
los datos volvieron completos. Si faltaban cambios recientes, se notó la diferencia
si faltaban

En teoría la sesión nos enseñó a auditar, hacer respaldos completos e incrementales y practicar la recuperación para no perder datos cuando algo sale mal.

Preguntas:

¿Por qué es importante practicar la recuperación antes de un fallo real?
¿Para qué sirve el trigger de auditoría
¿Qué guardó el trigger cuando se cambian las monedas
¿Qué comando se usa para hacer un respaldo total
¿Qué incluye el archivo generado por mysqldump?
¿Qué es el binary log (binlog)
¿Qué significa PITR
¿Por qué es importante poner fecha y hora en el nombre del respaldo
¿Qué se hace en el simulacro de desastre
¿Qué pasa si el respaldo no está actualizado

Elabore un resumen de la Sesión 5 destacando los aspectos más relevantes y los temas principales abordados

1.

Seleccione los trabajos terminados que ya se encuentran subidos a su repositorio:

- **☐ Sección 1.**
- **☐ Sección 2.**
- **☐ Sección 3.**
- **☐ Sección 4.**
- **☐ Sección 5.** echo
- **☐ Sección 6.**
- **☐ Sección 7.**
- **☐ Sección 8.**